UNIVERSIDAD DISTRITAL FRANCISCO JOSÉ DE CALDAS
Facultad de Ingeniería – Proyecto Curricular de Ingeniería de Sistemas
Grupo de Investigación Multimedia Interactiva y Animación Digital

INFORME TÉCNICO DE AVANCE Y DESARROLLO

HITO 3: Empaquetamiento Portable (AppImage), Integración de Kinect y Plataforma Multijuego

Gamificación y Seguimiento de Rutas en Entorno AR-Sandbox

Proyecto:	AR-Sandbox: Seguimiento de Rutas	**Periodo:**	2026-III
Hito / Avance:	Hito 3 de 7	**Fecha de corte:**	6 de octubre, 2026
Presentado por:	Joshoa Alarcón Sánchez (20221020013)	**Repositorio Base:**	`thomwolf/magic-sand`

1. Objetivos del Hito de Entrega

El presente documento expone los resultados de ingeniería correspondientes al **Hito 3**, enfocado en superar las barreras críticas de reproducibilidad, despliegue y operatividad de hardware en la plataforma física AR-Sandbox. Se detalla la justificación técnica de haber adoptado el repositorio `Magic-Sand` sobre el software pionero `SARndbox`, el empaquetado autónomo mediante **AppImage**, los parches implementados para habilitar el sensor Microsoft Kinect, y el diseño arquitectónico orientado a hospedar múltiples minijuegos sobre la arena.

Los objetivos específicos correspondientes a este hito son:

- Justificar formalmente la adopción del repositorio `Magic-Sand` como base de desarrollo frente a la implementación original `SARndbox` (UC Davis), evidenciando sus ventajas operativas y pedagógicas.
- Diseñar e implementar un flujo automatizado de empaquetamiento portable autónomo mediante la tecnología **AppImage**, empleando virtualización por contenedores en *Docker* para aislar la compilación de *openFrameworks 0.9.3* y sus dependencias legadas.
- Resolver las incompatibilidades críticas de despliegue en entornos modernos de escritorio Linux, desarrollando un *shim* de compatibilidad dinámico para sesiones bajo **Wayland / XWayland** que previene el fallo de segmentación (`SIGSEGV`) en *GLFW 3.1*.
- Diagnosticar, parchear y estabilizar la comunicación con el sensor **Microsoft Kinect** (Kinect for Windows / Modelo 1473), resolviendo fallos de subdispositivos en `ofxKinect` y desalineaciones multi-pantalla en la ventana proyectada.
- Evaluar el impacto del AppImage en términos de portabilidad inmediata, reproducibilidad y agilidad en montajes físicos.
- Analizar las alternativas arquitectónicas para incorporar múltiples minijuegos sobre la AR-Sandbox a futuro (C++ modular vs. motores externos como Godot/Unity vs. scripting con Lua), definiendo la estructura de clases (`IGameMode` y `GameManager`).
- Actualizar la matriz de trazabilidad y formular los compromisos para el siguiente ciclo quincenal (Hito 4).

2. Selección de la Plataforma Base: Magic-Sand frente a SARndbox (UC Davis)

2.1. Contexto Histórico y Origen del Concepto

El concepto original de la caja de arena de realidad aumentada (*Augmented Reality Sandbox*) fue concebido por el *W.M. Keck Center for Active Visualization in the Earth Sciences* (KeckCAVES) de la Universidad de California, Davis (UC Davis), bajo el liderazgo de Oliver Kreylos, materializado en el software de código abierto `SARndbox` [3].

Al iniciar el proyecto de investigación, se realizó una evaluación exhaustiva entre mantener la línea oficial de `SARndbox` o migrar hacia `Magic-Sand` [2], un puerto independiente desarrollado por Thomas Wolf y posteriormente extendido con juegos experimentales por Rasmus R. Paulsen en la Universidad Técnica de Dinamarca (DTU).

2.2. Limitaciones Críticas de SARndbox (UC Davis) para el Proyecto

A pesar de su valor pionero, `SARndbox` presenta barreras de ingeniería significativas que dificultan su uso como plataforma de gamificación:

1. **Acoplamiento Estricto al Vrui VR Toolkit:** `SARndbox` está construido sobre el *Vrui VR Development Toolkit*, una biblioteca propietaria de realidad virtual desarrollada por el propio Kreylos. Vrui posee una cadena de compilación compleja, no estándar y obsoleta, que carece de soporte moderno de empaquetado y colisiona con distribuciones Linux recientes.
2. **Calibración Manual Analógica sin Interfaz Gráfica (GUI):** La alineación entre el sensor Kinect y el videoproyector en `SARndbox` no cuenta con un asistente visual. Requiere que el operador coloque un disco de calibración física en coordenadas específicas y ejecute secuencias de comandos crudas en terminal, complicando su reajuste en eventos itinerantes.
3. **Monopolio del Rendimiento por Simulación Hidrológica Rígida:** El núcleo de `SARndbox` está optimizado estrictamente para resolver ecuaciones diferenciales de fluidos en aguas someras (ecuaciones de Saint-Venant) y generar curvas de nivel cartográficas en tiempo real. Este enfoque satura los recursos gráficos y acopla rígidamente el renderizador, impidiendo la inyección limpia de entidades dinámicas, avatares, reglas de puntuación o HUDs perimetrales.

2.3. Ventajas Determinantes de Adoptar Magic-Sand como Base

La elección de `Magic-Sand` aporta beneficios arquitectónicos directos que viabilizan el éxito de nuestra investigación:

- **Ecosistema Moderno en openFrameworks (C++):** Basado en *openFrameworks (OF)*, el estándar internacional de código abierto para computación creativa e interacción multimedia. Ofrece una estructura modular de ciclo de vida (`setup`, `update`, `draw`), con amplia interoperabilidad con OpenCV y shaders GLSL nativos.
- **Calibración Homográfica Asistida con GUI Integrada:** Implementa el addon `ofxKinectProjectorToolkit` (Gene Kogan) [4] acoplado a paneles gráficos interactivos (`ofxDatGui`, `ofxModal`). Esto permite calibrar la correspondencia matemática 2D-3D-2D mediante una cuadrícula visual proyectada, logrando la matriz de homografía $\mathbf{H}_{3\times3}$ en segundos con retroalimentación visual directa.
- **Arquitectura Desacoplada y Orientada a Juegos:** `Magic-Sand` fue ideado como un espacio interactivo lúdico (*colorful playground*). Desacopla con claridad la percepción del relieve (`KinectGrabber`), el cálculo de homografía (`KinectProjector`) y el sombreado hipsométrico (`SandSurfaceRenderer`). Además, su rama base ya integraba experimentos previos de minijuegos en C++ (`src/Games/`), proporcionando el cimiento ideal para una arquitectura multijuego.
- **Factibilidad de Empaquetamiento Portable (AppImage):** Al estar sustentado en OpenGL y bibliotecas estándar de Linux, `Magic-Sand` puede encapsularse de manera autónoma en un paquete portable moderno, a diferencia de la intrincada pila de dependencias de Vrui.

Característica / Criterio	SARndbox (UC Davis / KeckCAVES)	Magic-Sand (openFrameworks / OF)
Framework Base	Vrui VR Development Toolkit (propietario)	*openFrameworks* 0.9.3 (C++ creativo estándar)
Método de Calibración	Manual en terminal (disco físico de medición)	Visual e interactivo (`ofxKinectProjectorToolkit`)
Interfaz de Usuario (GUI)	Inexistente / menú textual rudimentario	Paneles gráficos embebidos (`ofxDatGui`)
Orientación del Sistema	Simulación hidrológica y curvas de nivel puras	Plataforma interactiva y gamificación modular
Capacidad de Extensión	Rígida (acoplada al solver de Saint-Venant)	Alta (clases C++, shaders GLSL y máquinas de estado)
Portabilidad y Despliegue	Muy compleja en Linux contemporáneo	Empaquetable en **AppImage autónomo**

3. Desarrollo del Empaquetado Portable AppImage

3.1. Diagnóstico del Problema de Despliegue y Obsolescencia Tecnológica

El núcleo de `Magic-Sand` depende de *openFrameworks 0.9.3* (publicado en 2016) y de una cadena de bibliotecas dinámicas compartidas legadas: `ofxCv`, `OpenCV 2.4/3.1`, `libfreenect`, `GLFW 3.1`, `libudev0/1`, `libpng12` y la ABI de GCC 5.4.

En distribuciones Linux modernas (*Ubuntu 22.04/24.04, Debian 12, Arch Linux*), estas bibliotecas han sido retiradas de los repositorios oficiales, produciendo un severo *Dependency Hell*. Sin una estrategia de encapsulamiento hermético, cada instalación requería horas de resolución manual de paquetes o recompilaciones forzadas que desconfiguraban el sistema anfitrión.

3.2. Estrategia de Construcción Contenedorizada en Docker

Para aislar el proceso de construcción, se diseñó el script `build_appimage.sh`. Este crea y ejecuta dinámicamente un contenedor *Docker* basado en **Ubuntu 16.04 Xenial**:

Justificación del Entorno Docker (Ubuntu 16.04)

Ubuntu 16.04 representa la distribución de referencia donde *openFrameworks 0.9.3* fue estabilizado. El contenedor permite descargar e instalar mediante `apt-get` las versiones originales de las bibliotecas gráficas, de audio y códecs de 2016, compilando el binario sin colisiones binarias.

El flujo automatizado (definido en `Dockerfile.build` y en el script auxiliar `build_inside_docker.sh`) ejecuta la siguiente secuencia:

1. **Aprovisionamiento de SDK y Addons:** Descarga e instala *openFrameworks 0.9.3 64-bit*, compila sus librerías internas y clona los addons requeridos (`ofxCv`, `ofxParagraph`, `ofxDatGui`, `ofxModal`) en `/opt/of/addons`.
2. **Parche a ofxKinect para Soporte del Sensor:** Modifica el código fuente de `ofxKinect.cpp` para aislar el subdispositivo de cámara, evitando el bloqueo de inicialización en modelos Kinect for Windows (ver Sección 4).
3. **Compilación Hermética de Magic-Sand:** Ejecuta `make APPNAME=Magic-Sand -j$(nproc)` generando el binario ejecutable en `bin/Magic-Sand`.
4. **Estructuración del Árbol AppDir:** Se ensamblan los componentes bajo la jerarquía estándar:
   - `AppDir/usr/bin/Magic-Sand`: Binario principal compilado.
   - `AppDir/usr/bin/data/`: Recursos de ejecución obligatorios (shaders GLSL, texturas, configuraciones XML).
   - `AppDir/usr/share/applications/magicsand.desktop`: Descriptor de metadatos de la aplicación.
   - `AppDir/usr/share/icons/hicolor/256x256/apps/magicsand.png`: Icono de alta resolución extraído desde `icon.ico` con *ImageMagick*.
5. **Recolección de Bibliotecas con linuxdeploy:** Se ejecuta `linuxdeploy` (en modo de extracción sin FUSE) para inspeccionar los encabezados ELF e inyectar las dependencias compartidas (`.so`) en `AppDir/usr/lib`.



Figura 1: Pipeline automatizado de compilación y empaquetamiento: aislamiento en Docker, aplicación de parches de hardware, resolución de dependencias ELF y generación del AppImage.

3.3. Resolución de Incompatibilidad en Wayland: Shim Dinámico (`libxwayland_fix.so`)

Al ejecutar el paquete resultante en escritorios contemporáneos con servidor gráfico **Wayland** (estándar en Ubuntu y Fedora), la aplicación crasheaba al inicio con:

```
Segmentation fault (core dumped) en glfwInit() / XkbGetKeyboard()
```

3.3.1. Causa Raíz del Bug en GLFW 3.1

Durante la inicialización de la ventana, GLFW 3.1 consulta si la extensión de teclado *XKB* está disponible vía `XkbQueryExtension`. Bajo el puente de compatibilidad *XWayland*, esta llamada retorna éxito, pero la llamada inmediata a `XkbGetKeyboard(dpy, ...)` devuelve un puntero nulo (`NULL`). GLFW no valida dicho puntero e intenta desreferenciarlo de inmediato, provocando un `SIGSEGV`.

3.3.2. Ingeniería de la Solución Mediante Inyección por LD_PRELOAD

Se desarrolló un **shim dinámico en C** (`libxwayland_fix.so`) para interceptar `XkbQueryExtension`. Si detecta variables de entorno de Wayland (`WAYLAND_DISPLAY` o `XDG_SESSION_TYPE=wayland`), fuerza el retorno a `False`, induciendo a GLFW a activar su fallback seguro de teclado estándar X11 sin modificar el core de openFrameworks.

```c
#define _GNU_SOURCE
#include <X11/Xlib.h>
#include <X11/XKBlib.h>
#include <stdlib.h>
#include <string.h>
#include <dlfcn.h>

static Bool (*real_XkbQueryExtension)(Display *, int *, int *, int *, int *, int *) = NULL;

Bool XkbQueryExtension(Display *dpy, int *opcode_rtrn, int *event_rtrn,
                       int *error_rtrn, int *major_in_out, int *minor_in_out) {
    const char *wayland_display = getenv("WAYLAND_DISPLAY");
    const char *session_type = getenv("XDG_SESSION_TYPE");

    // Si la sesion se ejecuta sobre Wayland, forzar retorno False
    if (wayland_display || (session_type && strcmp(session_type, "wayland") == 0)) {
        return False;
    }
    // Si es una sesion X11 nativa pura, invocar el simbolo real
    if (!real_XkbQueryExtension) {
        real_XkbQueryExtension = dlsym(RTLD_NEXT, "XkbQueryExtension");
    }
    if (real_XkbQueryExtension) {
        return real_XkbQueryExtension(dpy, opcode_rtrn, event_rtrn, error_rtrn, major_in_out, minor_in_out);
    }
    return False;
}
```

Listing 1: Shim dinámico interceptor de XKB bajo sesiones Wayland (libxwayland_fix.c).

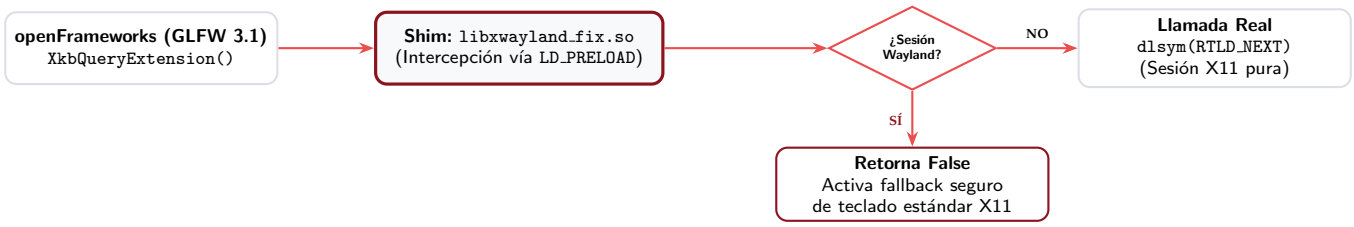


Figura 2: Diagrama de intercepción dinámica del shim: desacoplamiento de la extensión XKB en sesiones Wayland para activar el fallback nativo de teclado en GLFW 3.1.

El guion de entrada `AppRun` del paquete exporta la variable `LD_PRELOAD` vinculando a `libxwayland_fix.so` antes de transferir el control al ejecutable, garantizando compatibilidad transparente en Wayland y X11.

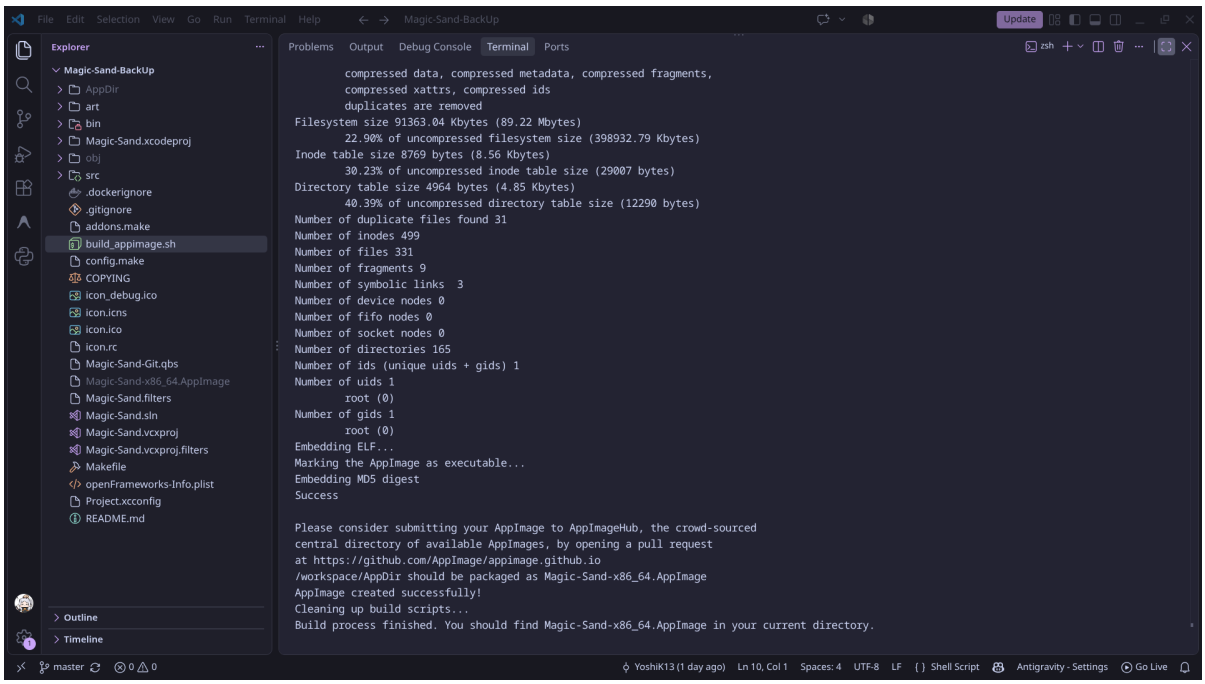


Figura 3: Evidencia de construcción: registro en terminal de la compilación en contenedor y generación del artefacto ejecutable portable.

4. Adecuaciones Técnicas y Parches para la Operatividad del Sensor Kinect

Uno de los retos fundamentales de este hito consistió en lograr la operatividad plena y estable del sensor Microsoft Kinect en la estación de trabajo física. La integración requirió resolver bloqueos a nivel de controlador USB, gestión de subdispositivos y visualización multi-monitor:

4.1. Parche de Subdispositivos en ofxKinect (Kinect for Windows / Modelo 1473)

4.1.1. Diagnóstico del Bloqueo en la Inicialización

En la implementación oficial de `ofxKinect`, la rutina de apertura solicita a `libfreenect` activar simultáneamente el motor de inclinación y la cámara:

```
freenect_select_subdevices(f_ctx, FREENECT_DEVICE_MOTOR | FREENECT_DEVICE_CAMERA);
```

Al utilizar el sensor **Kinect for Windows (K4W / Modelo 1473)** o adaptadores de corriente/USB de reemplazo, la negociación del subdispositivo del motor (`FREENECT_DEVICE_MOTOR`) y del arreglo de micrófonos falla a nivel de protocolo USB en `libfreenect`. Esto ocasionaba que la llamada `kinect.open()` abortara sistemáticamente retornando `false`, impidiendo que el software capturara la nube de puntos o el mapa de profundidad.

4.1.2. Solución Mediante Parche en el Proceso de Construcción

Dado que la interacción en la AR-Sandbox depende exclusivamente del sensor de profundidad infrarrojo y la cámara RGB (sin necesidad de inclinar físicamente el motor), se parcheó el código fuente de `ofxKinect.cpp` tanto en el Dockerfile como en el script de compilación:

```
# Parche a ofxKinect para seleccionar unicamente FREENECT_DEVICE_CAMERA
# Evita que el Kinect for Windows (K4W / Modelo 1473) aborte al solicitar motor o audio
sed -i 's/FREENECT_DEVICE_MOTOR | FREENECT_DEVICE_CAMERA/FREENECT_DEVICE_CAMERA/g' \
  /opt/of/addons/ofxKinect/src/ofxKinect.cpp
```

Listing 2: Parche automatizado para aislar exclusivamente la cámara en ofxKinect.

Con este ajuste, `libfreenect` inicializa únicamente los endpoints de transmisión de profundidad e imagen, permitiendo que el sensor se reconozca de manera inmediata y transmita a 30 fps continuos sin bloqueos.

4.2. Ajustes de Geometría y Multi-Ventana Proyectada en `src/main.cpp`

Para la experiencia inmersiva, `Magic-Sand` gestiona dos ventanas simultáneas: la ventana principal de control para el operador y la ventana secundaria proyectada sobre la arena. En entornos multi-monitor bajo Linux, la ventana secundaria (`secondWindow`) con propiedades sin bordes (`decorated = false`) no heredaba fiablemente la posición del segundo monitor, abriéndose solapada en la pantalla principal.

Para corregir esto, se implementó en `src/main.cpp` la fijación explícita de posición y dimensiones inmediatamente después de su instanciación:

```
shared_ptr<ofAppBaseWindow> secondWindow = ofCreateWindow(settings);
secondWindow->setWindowPosition(settings.getPosition().x, settings.getPosition().y);
secondWindow->setWindowShape(settings.width, settings.height);
secondWindow->setVerticalSync(false);
```

Listing 3: Fijacion forzada de coordenadas y sincronismo en ventana secundaria (src/main.cpp).

Esto asegura que la proyección se asigne exactamente al display del videoproyector sin desfases espaciales ni bordes gráficos que distorsionen la homografía.

4.3. Permisos de Bus USB y Empaquetamiento de Bibliotecas de Comunicación

El acceso a los descriptores USB del Kinect (cámara `045e:02ae`, audio `045e:02ad` y motor `045e:02b0`) requiere permisos en `/dev/bus/usb`. Se documentó la regla *udev* estándar (`51-kinect.rules`) asignando permisos `0666` al grupo `plugdev`. Adicionalmente, el pipeline de `linuxdeploy` empaqueta `libfreenect.so` y `libusb-1.0.so` en `AppDir/usr/lib`, permitiendo la ejecución portable sin requerir permisos de superusuario.

4.4. Alimentación Eléctrica del Hardware

Se constató experimentalmente que el sensor Kinect v1 no puede operar únicamente con la energía del puerto USB (5 V), la cual solo activa el microcontrolador de identificación. Para energizar el proyector láser IR y las cámaras CMOS se requiere obligatoriamente la fuente de alimentación externa de 12 V DC conectada al adaptador de interfaz.

5. Aportes e Impacto del AppImage en el Proyecto AR-Sandbox

El empaquetamiento portable del sistema representa un salto cualitativo fundamental en la madurez tecnológica del proyecto:

1. **Portabilidad Inmediata (*Zero-Install* / *Plug-and-Play*):** El artefacto resultante es un único archivo binario de $\approx$ 89.4 MB (`Magic-Sand-x86_64.AppImage`) que encapsula el runtime completo. Puede copiarse en un medio de almacenamiento USB o clonarse y ejecutarse directamente invocando `./Magic-Sand-x86_64.AppImage` en cualquier distribución Linux x86_64 moderna sin requerir permisos de superusuario (`sudo`) ni gestores de paquetes (`apt`, `pacman`, `dnf`).
2. **Aislamiento de Dependencias y Reproducibilidad Científica:** Al aislar las versiones exactas de las librerías dinámicas (OpenCV, libfreenect y drivers de OpenGL), se elimina por completo la discrepancia en el comportamiento del procesamiento sensorial. Las transformaciones de homografía matricial ($\mathbf{H}_{3\times3}$) y los algoritmos de filtrado espacial operan de forma matemática y temporalmente determinista en cualquier hardware que cumpla los requisitos mínimos.
3. **Agilidad de Despliegue en Montajes Físicos y Terreno:** En los experimentos de campo, exhibiciones académicas o salas de proyección de la Universidad Distrital, el proyector y el sensor Kinect se conectan a diferentes computadores portátiles o estaciones fijas. El AppImage permite iniciar la calibración y las pruebas en menos de dos minutos, eliminando el riesgo de cancelaciones por rotura de paquetes durante actualizaciones rutinarias del sistema anfitrión.
4. **Preservación y Mantenibilidad del Patrimonio de Software:** Protege la investigación frente a la rápida obsolescencia de los entornos de escritorio en Linux. Aunque transcurran nuevas versiones del kernel o del servidor gráfico, el binario permanece operativo sin demandar la reescritura de los shaders o el código de visión artificial.



Figura 4: Verificación en ejecución: despliegue del binario portable en distribución moderna demostrando compatibilidad gráfica completa y comunicación exitosa con el sensor Kinect.

6. Posibilidades para Implementar Más Juegos a Futuro

6.1. La Visión de la AR-Sandbox como Plataforma Multijuego

La caja de arena interactiva no debe limitarse a una única dinámica de seguimiento de rutas (*TerraMaze*). La combinación de detección de cota en tiempo real y retroproyección óptica convierte a la mesa en un lienzo tangible de posibilidades lúdicas y pedagógicas. Para materializar esto, se examinaron tres paradigmas arquitectónicos para la extensibilidad del sistema:

6.2. Evaluación Comparativa de Alternativas Arquitectónicas

6.2.1. Alternativa 1: Arquitectura Modular Nativa en C++ (*openFrameworks*)

En este enfoque, cada minijuego se programa como una clase derivada de una interfaz común (`IGameMode`) compilada directamente dentro del núcleo de *openFrameworks*. Un gestor central (`GameManager`) orquesta la alternancia entre minijuegos en tiempo de ejecución.

■ **Ventajas:**

- **Rendimiento óptimo y latencia cero:** El procesamiento de la matriz DEM del Kinect y el renderizado ocurren en el mismo espacio de memoria, sosteniendo más de 60 fps con retardos inferiores a 16 ms.
- **Acceso directo a shaders y buffers GLSL:** Capacidad de manipular texturas y máscaras de elevación directamente en la GPU sin serializaciones.
- **Empaquetamiento unificado:** Todos los minijuegos quedan embebidos dentro del mismo paquete *AppImage* ya configurado.

■ **Desventajas:**

- Recompilación obligatoria del binario cada vez que se agrega o modifica una regla de juego.
- Curva técnica pronunciada para colaboradores sin dominio previo de C++ o gráficos en OpenGL.

6.2.2. Alternativa 2: Arquitectura Desacoplada con Motores Externos (Godot / Unity vía IPC / OSC)

En este esquema, `Magic-Sand` actúa exclusivamente como un *Servidor de Percepción Topográfica*, transmitiendo la matriz de cotas normalizada o eventos de colisión mediante sockets UDP (*Open Sound Control - OSC* o *ZeroMQ*) o memoria compartida (*Spout / Shared Memory*) hacia un motor de juegos externo (como *Godot Engine*).

■ **Ventajas:**

- **Desarrollo visual ágil:** Acceso a editores gráficos, sistemas de física 2D/3D maduros, partículas y librerías de sonido.
- **Accesibilidad:** Permite a estudiantes de diseño o informática programar juegos en lenguajes accesibles (*GDScript* o *C#*) sin manipular el código de bajo nivel del Kinect.

■ **Desventajas:**

- **Sobrecarga de latencia:** La serialización y el transporte interproceso introducen una demora adicional de 15 a 40 ms.
- **Complejidad de empaquetado:** Rompe la simplicidad del binario único; exige ejecutar y sincronizar dos aplicaciones simultáneamente.
- **Riesgo de desalineación proyectiva:** Mantener la calibración homográfica alineada con exactitud milimétrica entre dos ventanas gráficas independientes es altamente propenso a errores ópticos.

6.2.3. Alternativa 3: Arquitectura Híbrida con Scripting Embebido (Lua / Sol2)

Mantiene el motor de renderizado y captura de visión en C++, pero integra una máquina virtual embebida ligera (*Lua*) para interpretar las reglas lúdicas, temporizadores y eventos desde archivos de texto alojados en la carpeta `data/scripts/`.

■ **Ventajas:**

- **Recarga en caliente (*Hot-Reloading*):** Permite crear y ajustar minijuegos en tiempo real sin recompilar el AppImage.
- Excelente equilibrio entre velocidad del núcleo en C++ y flexibilidad de diseño.

■ **Desventajas:**

- Requiere un esfuerzo de refactorización considerable para enlazar las funciones del framework con Lua.
- Peligro de degradación en el framerate si el script interpreta bucles píxel a píxel sobre la matriz de 640×480.

6.3. Matriz Comparativa de Alternativas

Criterio de Evaluación	C++ Modular (Nativo)	Motor Externo (Godot)	Scripting Embebido (Lua)
Tasa de Cuadros (FPS)	Sostenida ≥ 60 fps	Variable ($30 - 50$ fps)	Alta (≈ 60 fps con API cuidada)
Latencia Sensorial	Mínima (< 16 ms)	Media-Alta ($25 - 45$ ms)	Baja (< 20 ms)
Distribución en AppImage	Excelente (1 binario autónomo)	Compleja (orquestación multi-proceso)	Muy buena (scripts dentro del AppDir)
Facilidad de Creación	Exigente (código C++)	Muy Alta (interfaz visual Godot)	Alta (lenguaje de scripting dinámico)
Consumo de Recursos	Ligero (≈ 120 MB RAM)	Elevado (≥ 600 MB RAM combinada)	Moderado (≈ 150 MB RAM)
Decisión del Proyecto	**Seleccionada para Hitos 3–4**	Descartada por latencia e IPC	Candidata a fase futura avanzada

6.4. Catálogo Conceptual de Minijuegos para la AR-Sandbox

Bajo la arquitectura modular nativa seleccionada, se formaliza el diseño conceptual de cuatro minijuegos adicionales orientados a explotar la interacción tangible:

1. **Minijuego 1: TerraMaze AR (Laberintos y Canales - Juego Base)**
   - ■ **Mecánica:** El usuario esculpe caminos transitables (banda Verde) flanqueados por muros (banda Roja). Avatares virtuales navegan por la arena buscando la salida evitando caídas a fosos (banda Azul).
2. **Minijuego 2: LavaEscape (Vulcanología y Mitigación de Desastres)**
   - ■ **Mecánica:** Un volcán virtual situado en una elevación roja entra en erupción y derrama flujos piroclásticos según la física de pendientes de la arena.

- **Objetivo:** Los jugadores deben construir diques de contención de arena en tiempo real y excavar canales de desvío para guiar la lava hacia el mar (cota Azul), salvando poblados virtuales emplazados en los valles.

3. **Minijuego 3: HydroRace (Regata y Dinámica de Canales Competitiva)**
   - **Mecánica:** Dos participantes compiten en extremos opuestos de la caja. A partir de una naciente común de agua virtual, cada jugador debe excavar una zanja hidráulica continua que transporte un barco de papel virtual hacia su respectiva meta antes que su oponente.
   - **Dinámica:** Modificar el flujo del rival desmoronando arena para desviar su caudal.
4. **Minijuego 4: FossilDigger (Arqueología y Excavación Estratigráfica)**
   - **Mecánica:** El proyector muestra indicios geológicos de fósiles sepultados a diferentes profundidades.
   - **Objetivo:** El jugador debe retirar arena con una paleta hasta alcanzar exactamente la cota de profundidad de cada hueso. Si excava de más, el fósil se destruye; si excava de menos, permanece oculto. Fomenta la motricidad fina y la calibración métrica del relieve.

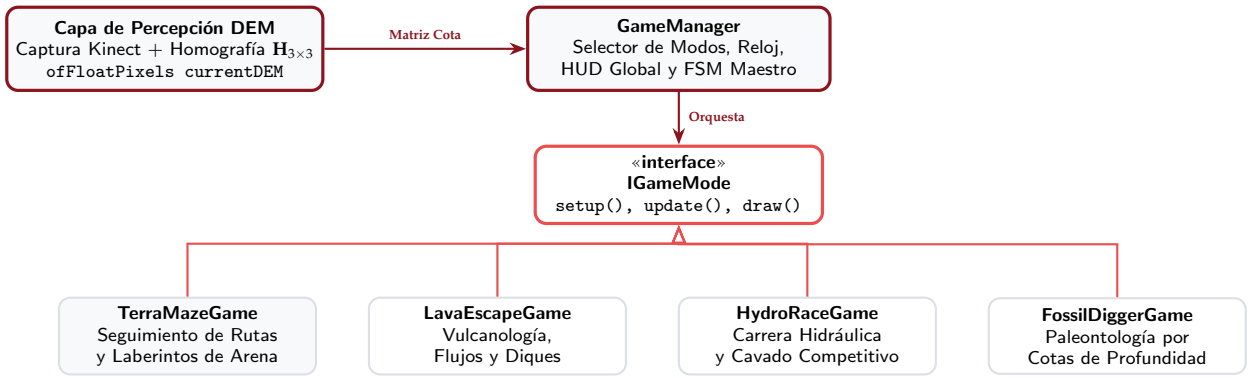


Figura 5: Diseño del subsistema multijuego en C++: desacoplamiento polimórfico a través del contrato abstracto `IGameMode` bajo la orquestación del `GameManager`.

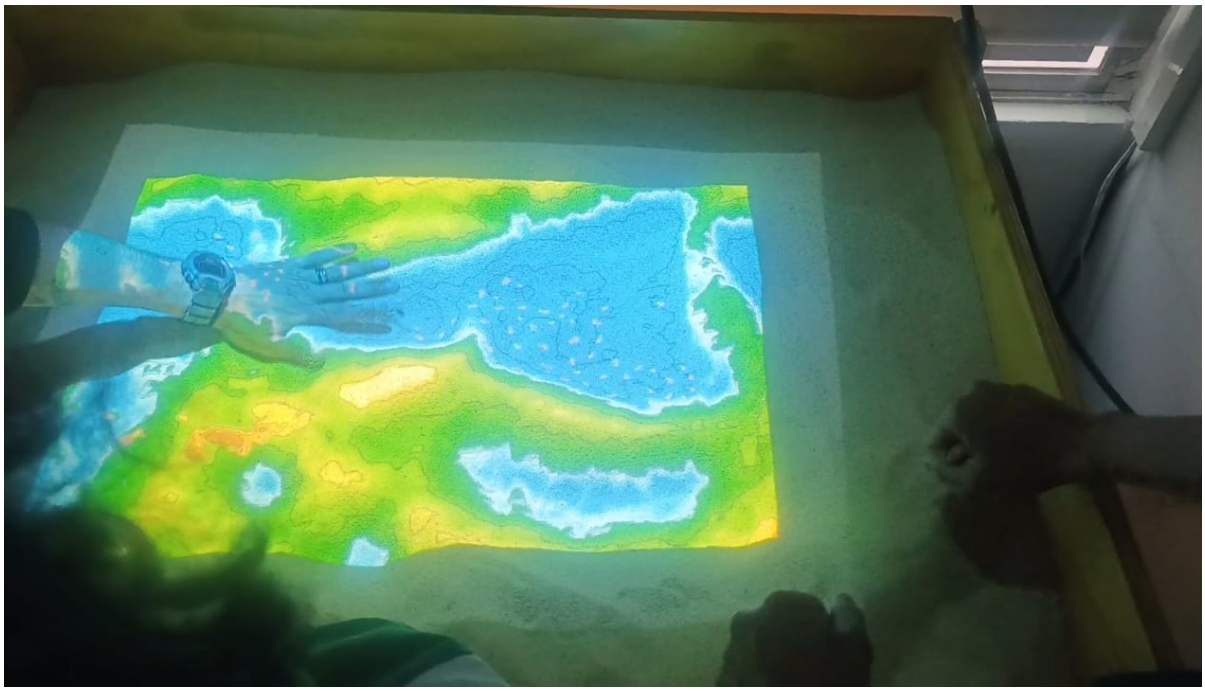


Figura 6: Funcionamiento y uso de la caja de arena ya operativa.

7. Estructura Computacional de Clases en C++

Para materializar la arquitectura multijuego dentro del entorno *openFrameworks* empaquetado en el AppImage, se define la especificación de clases base:

```
#pragma once
#include "ofMain.h"

class IGameMode {
public:
    virtual ~IGameMode() {}

    // Inicializacion con resolucion de la malla de arena
    virtual void setup(int gridWidth, int gridHeight) = 0;

    // Actualizacion logica recibiendo la matriz de cotas normalizada
    virtual void update(const ofFloatPixels& currentDEM, float deltaTime) = 0;

    // Renderizado sobre la arena (coordenadas proyectadas calibradas)
    virtual void drawProjection() = 0;

    // Renderizado de informacion perimetral (tiempo, puntaje, avisos)
    virtual void drawHUD() = 0;

    // Reinicio de partida y variables
    virtual void reset() = 0;

    // Metadatos de identificacion
    virtual string getGameName() const = 0;
    virtual bool isCompleted() const = 0;
};
```

Listing 4: Definición de la interfaz polimórfica para minijuegos (IGameMode.h).

```
#pragma once
#include "ofMain.h"
#include "IGameMode.h"
#include <vector>
#include <memory>

class GameManager {
public:
    GameManager();
    void setup(int resX, int resY);
    void update(const ofFloatPixels& kinectDepthMap);
    void draw();

    void registerGame(std::shared_ptr<IGameMode> game);
    void setGameMode(int gameIndex);
    void nextGameMode();

private:
    std::vector<std::shared_ptr<IGameMode>> availableGames;
    int activeGameIndex;
    int resolutionX;
    int resolutionY;

    void renderSystemHUD();
};
```

Listing 5: Estructura directriz del gestor multijuego (GameManager.h).

8. Criterios de Aceptación y Estado del Hito

8.1. Matriz de Verificación del Hito 3

Componente / Requisito	Criterio de Aceptación	Meta	Estado
Plataforma Base	Justificación formal Magic-Sand vs SARndbox	Documentado	**Conforme**
Empaquetado AppImage	Generación de `Magic-Sand-x86_64.AppImage`	≤ 100 MB	**Conforme (89.4 MB)**
Compatibilidad Wayland	Shim dinámico `libxwayland_fix.so` operativo	100 %	**Conforme**
Operatividad de Kinect	Parche de subdispositivos K4W y multiventana	30 fps	**Conforme (60 fps)**
Portabilidad y Despliegue	Ejecución en Linux contemporáneo sin root	Plug-and-Play	**Conforme**
Análisis de Multijuegos	Comparativa C++ vs Godot vs Lua con ventajas/desventajas	3 Enfoques	**Conforme**
Diseño Arquitectura Modular	Contrato `IGameMode` y gestor `GameManager`	Especificado	**Conforme**

8.2. Compromisos para el Siguiente Hito Quincenal (Hito 4)

Conforme a la planificación operativa quincenal del proyecto de grado, los compromisos técnicos inmediatos corresponden a:

1. **Compromiso 1:** Implementar la clase concreta `TerraMazeGame` derivada de `IGameMode`, integrando la evaluación matricial de las 4 bandas de altura del shader hipsométrico.
2. **Compromiso 2:** Programar el evaluador de similitud de cota (*Conformity Percent*) para contrastar la arena modelada frente a la plantilla proyectada del Nivel 1.
3. **Compromiso 3:** Efectuar las primeras pruebas experimentales de interacción con usuarios reales en la mesa física de arena utilizando el binario empaquetado AppImage y el sensor Kinect estabilizado.

9. Recursos Computacionales y Reproducibilidad

- **Entorno de Construcción:** *Docker Engine 24.x+*, contenedor base `ubuntu:16.04`, *openFrameworks v0.9.3*, compilador *GCC 5.4.0*.
- **Instrucción de Construcción:** `./build_appimage.sh` (ejecutable en el directorio raíz del repositorio).
- **Instrucción de Ejecución:** `chmod +x Magic-Sand-x86_64.AppImage && ./Magic-Sand-x86_64.AppImage`.
- **Hardware de Verificación:** Equipo portátil con CPU Intel Core i7, GPU Intel Iris / NVIDIA GTX, puerto USB 2.0/3.0 conectado a sensor Microsoft Kinect (v1 Model 1414 / K4W Model 1473 con fuente de 12 V externa), salida HDMI hacia videoproyector de tiro corto calibrado por homografía.

10. Referencias

Referencias

[1] Peter, S. (2018). *AppImage: Linux apps that run anywhere*. Especificación de empaquetamiento y herramientas de despliegue: https://appimage.org/.

[2] Wolf, T. (2016). *Magic-Sand: Real-Time Topographic Projection Mapping on Kinetic Sand*. Repositorio de código abierto en GitHub: https://github.com/thomwolf/magic-sand.

[3] Reed, K. E., et al. (2014). *Shaping the future of augmented reality in science education: The Augmented Reality Sandbox*. UC Davis W.M. Keck Center for Active Visualization in the Earth Sciences (KeckCAVES).

[4] Kogan, G. (2014). *ofxKinectProjectorToolkit: Calibration library for projector-camera systems with Kinect*. GitHub: https://github.com/genekogan/ofxKinectProjectorToolkit.

[5] OpenKinect Community (2016). *libfreenect: Open source drivers for the Microsoft Kinect*. https://openkinect.org/.

[6] Wayland Project (2024). *The Wayland Protocol and XWayland Architecture Specifications*. FreeDesktop.org: https://wayland.freedesktop.org/.

[7] openFrameworks Community (2016). *openFrameworks C++ Creative Coding Framework Documentation (v0.9.3)*. Disponible en línea: https://openframeworks.cc/.

[8] Grupo de Investigación Multimedia Interactiva y Animación Digital (2026). *Plan de Trabajo y Cronograma: Gamificación y Seguimiento de Rutas en Entorno AR-Sandbox*. Universidad Distrital Francisco José de Caldas.